

Relatório Técnico Consolidado — Massa Hub / FeelWorks

Gerado a partir do contexto acumulado do projeto (CLAUDE.md, AGENTS.md, specs, migrations, handoffs de sessão) para avaliação de plataforma de desenvolvimento Open-Source (Low-Code/AI-Agentic).

1. Visão Geral e Escopo do Projeto

Objetivo principal: Massa Hub (marca de produto: FeelWorks) é a camada de infraestrutura profissional e reputação da creator economy brasileira — uma rede vertical B2B/B2B2C onde assessores, agências, profissionais criativos e marcas descobrem, contratam e comprovam trabalho real com criadores. Não é rede social de creators, não é marketplace de freelancer, não é ferramenta de gestão isolada (tipo Tãmi) nem plataforma fechada de matching marca-creator (tipo Squid/BrandLovrs). O produto ocupa deliberadamente uma camada que hoje não existe no mercado brasileiro: identidade profissional + prova de trabalho + reputação verificável do ecossistema.

O núcleo funcional da Fase 1 (atual) é o **módulo de oportunidades**: um assessor cola um briefing bruto (texto de WhatsApp, e-mail, áudio transcrito), uma IA extrai e estrutura os campos automaticamente, o humano revisa e publica um "ticket" de campanha que entra num mercado de duas colunas (lista + detalhe, estilo GitHub Issues/LinkedIn Recruiter) visível a perfis compatíveis.

Usuários finais — é um portal comercial/público, não uma ferramenta interna:

- **Primário e prioritário:** assessores de creators (2-3+ anos de experiência, hoje operam via WhatsApp). É o "cavalo de Troia" — wedge estratégico que desbloqueia os outros públicos.
- **Secundário:** pequenas agências (2-5 pessoas); profissionais criativos (foto, vídeo, design, jurídico, áudio); marcas contratantes; os próprios creators (perfil gratuito, nunca pagam).
- Multi-tenant conceitual: cada assessor/agência opera como um "workspace" que gerencia suas próprias oportunidades e relacionamentos — hoje simplificado (autoria por `autor_id`), não implementado como multi-tenancy formal.

Modelo de negócio: Fase 1 é 100% gratuita (foco em densidade e hábito). Monetização a partir da Fase 2, nesta ordem de prioridade: (1) assinatura de assessores/agências, (2) clube de benefícios via revenue share com parceiros (ex.: Noodle), (3) assinatura premium de creators. **Não há e não haverá:** fee transacional sobre campanhas como linha principal, publicidade, venda de dados, ou destaque pago no feed — decisões explícitas de posicionamento (conflitam com a proposta de reputação verificável e transparente).

Estágio atual: produto em construção ativa, pré-lançamento comercial, ~19 perfis povoados manualmente, módulo de oportunidades funcional em ambiente de produção mas com IA ainda não "destravada" por falta de créditos de API. Deadline de contexto: YOUNIX Summit, 29/set/2026 (evento do setor em São Paulo).

2. Arquitetura de Dados e Backend

Modelagem de dados: relacional, com regras de domínio não-triviais (não é CRUD simples nem conexão superficial a bancos existentes). O projeto segue **arquitetura hexagonal (ports & adapters) auditada e levada a sério**, não como jargão — é uma decisão constitucional documentada e verificada por regra de lint/review:

- `src/domain/` — entidades e value objects **puros**, zero import de framework, banco ou SDK externo. Testável sem rede e sem banco.
- `src/application/` — casos de uso, orquestra o domínio via portas (interfaces).
- `src/infrastructure/` — adapters concretos (Supabase, Anthropic).
- Regra dura, verificada em code review: **domínio nunca importa de** `infrastructure/`; `app/` (rotas Next.js) nunca importa adapter direto, só via caso de uso.
- Ordem de construção obrigatória: modelar domínio + testes primeiro, schema de banco depois. "Se o schema vier antes, o domínio vira refém do banco."

Entidades de domínio centrais (já modeladas e testadas):

- **Prova** — registro de trabalho real, imutável, assinado bilateralmente (criador + contratante). Status `em_andamento | verificada` é **derivado** (nunca atribuído manualmente); vira `verificada` só com ≥ 2 assinaturas de partes distintas.
- **Assinatura** — value object: `autorId`, `papel` (criador | contratante), `assinadoEm`.
- **Lastro** — value object de reputação: agregado de **fatos contáveis** (nº de provas verificadas, marcas distintas, marcas recorrentes, nº de recomendações). **Decisão de arquitetura inegociável (D1): jamais um score numérico ou nota composta.** Qualquer sugestão de plataforma que force reputação para um campo de "rating" numérico entra em conflito direto com uma regra de produto já testada e revertida uma vez.
- **Oportunidade** — entidade com squad multi-papel (`Papel { funcao, qtd }`), não vaga única genérica), campos estruturados (marca, budget, prazo, local, entregáveis, `perfilBuscado`), status `aberta | em_selecao | fechada`, e um bloco `origem` que rastreia se o ticket foi estruturado por IA e revisado por humano.
- **TicketExtraído** — value object intermediário (proposta da IA antes de virar Oportunidade), com campo `confianca` (0-1, qualidade de extração — deliberadamente um namespace separado do Lastro, para não confundir "confiança de extração de IA" com "score de reputação de pessoa", o que violaria D1).

Banco de dados: PostgreSQL via **Supabase**, usado estritamente como *adapter de persistência na borda* — não como fonte de verdade do modelo de domínio. Migrations versionadas em SQL puro (`supabase/migrations/*.sql`), aplicadas via `supabase db push`, nomenclatura cronológica, nunca editadas retroativamente (nova migration em vez de alterar uma aplicada). Tabelas atuais: `provas`, `assinaturas` (FK + unicidade por `prova_id + autor_id`), `oportunidades`. Índices explícitos por `autor_id`, `status`, `criado_em DESC`, `criador_id`, `prova_id`.

Row Level Security (RLS): habilitado em todas as tabelas, hoje em modo *deny-by-default* — nenhuma política pública aberta. Todo acesso da aplicação passa por `(service role)` server-side (bypassa RLS), com identidade do usuário validada no servidor via `(getUser())` do Supabase Auth antes de qualquer escrita (nunca confia em ID vindo do client/FormData). Dívida técnica registrada e consciente: políticas reais ancoradas em `(auth.uid() = autor_id)/(criador_id)` ainda não foram criadas porque, na arquitetura atual (tudo via service role), ficariam sem efeito — só fazem sentido após um refactor para leitura/escrita via cliente autenticado (SSR), planejado mas não-urgente.

Requisito de arquitetura para a plataforma avaliada: precisa permitir isolar regra de negócio de infraestrutura (não pode ser uma ferramenta que amarra lógica de domínio ao schema de banco ou a um builder visual proprietário). Precisa aceitar Postgres/Supabase como backend ou permitir trocar de backend sem reescrever regras de negócio — essa portabilidade é princípio ativo do projeto ("evita ficar refém do fornecedor").

3. Fluxos de Trabalho e Integrações (Workflows)

Sistemas de terceiros consumidos hoje:

- **Anthropic API** (Claude) — único uso de IA em produção: extração estruturada de ticket a partir de texto livre. Modelo em uso: `(claude-haiku-4-5-20251001)`. Chave só server-side (`(ANTHROPIC_API_KEY)`), nunca exposta ao client.
- **Google OAuth** via Supabase Auth — único método de login. Configurado manualmente no Google Cloud Console (OAuth consent externo) e no painel Supabase.
- **Vercel** — build e hosting; deploy automático a cada push no branch de produção `(main)`.

Fluxos de aprovação humana: o produto é desenhado deliberadamente para **nunca publicar automaticamente sem revisão humana**. É regra de produto, não só técnica: a IA propõe (extrai o ticket), o humano confirma/edita, só então publica. Isso vale também para o campo `(confianca)` — sinaliza briefing raso *antes* da publicação, não bloqueia sozinho.

Automações baseadas em evento: nenhum webhook ou cron job em produção hoje. O pipeline é síncrono e sob demanda: usuário cola texto → uma chamada à API Anthropic → validação de schema → preview editável → publicação manual. Não há fila, não há processamento assíncrono, não há jobs agendados. Isso é uma decisão consciente de escopo (ver seção 4), não uma limitação técnica ainda não resolvida.

Integrações previstas mas não construídas (roadmap Fase 2/3, não bloqueantes para a escolha de plataforma agora, mas relevantes para extensibilidade futura): parceria de reputação com Korun (modelo co-branded, não white-label); parceria de benefícios com Noodle (fintech); candidatas de Fase 3 como Afilio/Você.com.br; possível integração de atribuição de conversão física→digital vinda do braço FeelMakers (QR/NFC em espaço físico, ainda em fase de teste empírico, não de produto).

Requisito para a plataforma avaliada: precisa suportar chamadas de API externas (REST, uma por vez, síncronas) atrás de um adapter — não precisa de orquestração de workflow complexa, fila de mensagens, nem automação low-code de múltiplos passos. Simplicidade aqui é decisão de produto, não lacuna a preencher pela ferramenta.

4. Capacidades de IA e Sistemas Agênticos

Este é o ponto onde o projeto tem a política mais restritiva e mais explicitamente documentada — importante para não escolher uma plataforma que empurre para mais automação agêntica do que o projeto quer.

O que a IA faz hoje (e é o único uso permitido na Fase 1): uma tarefa, um agente, uma chamada. Colar briefing bruto → devolver JSON estruturado e validado (marca, budget, prazo, local, entregáveis, squad com papéis/quantidade/nicho/faixa de seguidores, nível de confiança, trechos de origem por campo para "realce auditável" na UI). A saída do modelo **nunca é confiada cegamente** — passa por validação de schema no adapter antes de virar objeto de domínio (`TicketExtraído`). Não há RAG, não há base de conhecimento vetorial, não há decisão autônoma de negócio tomada pela IA.

Decisão de arquitetura formalmente registrada (D10) e reafirmada após revisão consciente em 21/06/2026: sistema multi-agente está **em quarentena** na Fase 1.

Justificativa explícita do fundador: "multi-agente resolve escala que não existe com ~19 perfis; é a mesma sereia do Identity OS com roupa nova." Pipeline determinístico e testável é preferido a enxame autônomo. Critérios objetivos definidos para reavaliar (não "nunca mais", mas não automático): (a) volume de oportunidades/mês processado pelo agente único que torne o gargalo de precisão/cobertura visível, não de throughput de cadastro; (b) existência de features de produto que dão o que orquestrar (matching por Lastro, verificação cruzada, negociação) — hoje nenhuma existe; (c) plateau de qualidade mensurado no dataset de eval do agente único.

Human-agent handoff: binário e não-negociável. IA propõe → humano revisa cada campo → humano publica. Não há modo "auto-publish" nem confiança suficiente delegada à IA para pular a revisão, independentemente do score de confiança retornado.

Avaliação de qualidade da extração: existe um harness de eval (`eval/ticket/`), fixtures de briefing real + gabarito esperado, script determinístico de comparação de campos — não um segundo LLM julgando o primeiro). Não usa framework de otimização automática de prompt (ex. DSPy/MIPROv2) — o "otimizador" hoje é o fundador lendo divergências e ajustando o system prompt manualmente. Critério definido para justificar automação futura: dezenas a centenas de fixtures, não dois ou três casos.

Uso de IA fora do produto: o próprio desenvolvimento é feito com **Claude Code** como "equipe de execução" — o fundador não escreve código diretamente, orquestra sessões de agente de código contra uma constituição documentada (`CLAUDE.md`, `AGENTS.md`, `docs/specs/`, `docs/decisoões/log.md`). Isso é meta-relevante para a escolha de plataforma: o fluxo de trabalho real de desenvolvimento já é "prompt para um agente de código que lê contexto documentado e aplica mudanças", então uma plataforma candidata precisa ser

compatível com esse modo de trabalho (repositório Git real, não estado preso a um builder proprietário sem exportação de código).

Requisito para a plataforma avaliada: precisa suportar (a) uma porta de saída para chamada de LLM com validação de schema na resposta, sem forçar orquestração multi-agente; (b) fluxo de revisão humana obrigatória antes de qualquer ação de escrita irreversível; (c) não deve empurrar para RAG, memória vetorial ou agentes autônomos — isso está explicitamente fora do escopo atual e sua presença como "capacidade padrão" da plataforma seria um desalinhamento de complexidade, não um ganho.

5. Interface do Usuário (UI/UX)

Nível de complexidade exigido: telas customizadas e com identidade visual própria — não é um painel interno onde um builder de arrastar-e-soltar genérico resolveria. O produto tem um sistema de design documentado como fonte única de verdade (tokens em `globals.css`, Tailwind v4) e uma tese de direção de marca ativa ("Lastro × Ritmo — um registro que pulsa"), com princípios generativos de design formalizados (evidência sobre afirmação, o trabalho é o herói, densidade com respiro, mono como assinatura de dado, movimento que prova e não decora).

Stack de frontend hoje: Next.js 15 (App Router), React Server Components para leitura, Server Actions para escrita, Tailwind v4 via `@theme`. Tipografia: Archivo (display, 700–900, comprimida), Inter (corpo), IBM Plex Mono (dados, handles, contadores — vocabulário deliberadamente "GitHub", não SaaS arredondado). Paleta: paper off-white (`#FAFAF6`), tinta quase-preta (`#16151D`), violeta de marca (`#6C5BFF`), âmbar de destaque parcimonioso.

Padrões de UI específicos que já são contrato de produto, não sugestão:

- Workspace de dois painéis (lista + detalhe) no desktop para o mercado de oportunidades, estilo GitHub Issues/LinkedIn Recruiter — explicitamente não um "quadro passivo de vagas".
- "Realce de origem": trechos do texto bruto que originaram cada campo extraído ficam sublinhados na cor do campo correspondente na tela de revisão — mecanismo de auditabilidade da IA, tratado como "a assinatura de confiança da tela", não pode ser cortado por simplificação.
- Heatmap de contribuição (Ritmo) só renderiza com volume real de dados; estado vazio é tratado como comunicação ativa, não ausência.
- Regras de responsividade auditadas: `min-width: 0` em filhos de grid (bug nº1 encontrado), tab bar inferior fixa no mobile, modais viram bottom sheet, alvos de toque $\geq 44\text{px}$, textarea com `font-size: 16\text{px}` (evita zoom automático no iOS), auditoria de overflow horizontal em 360/390px a cada mudança de layout.

Requisito para a plataforma avaliada: precisa suportar CSS/design tokens customizados e componentização real (não apenas templates pré-definidos com theming superficial). Um construtor visual rápido de painéis internos **não atende** o requisito de frontend deste projeto — a interface pública é o produto, não uma tela de administração.

6. Requisitos Não-Funcionais e Infraestrutura

Onde roda: 100% cloud/SaaS gerenciado, deliberadamente — Vercel (build + hosting, região implícita via edge) + Supabase (Postgres, projeto isolado em `sa-east-1`), São Paulo, custo atual R\$ 0/mês no tier gratuito). Não há requisito de soberania de dados on-premise, não há Docker/Kubernetes no pipeline atual, e não há indicação de que isso mudará — o perfil é bootstrap de fundador solo, otimizando para menor operação possível, não para controle de infraestrutura.

Variáveis de ambiente / segredos geridos: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` (públicas), `SUPABASE_SERVICE_ROLE_KEY`, `ANTHROPIC_API_KEY` (server-only, nunca expostas ao client). O build do Next.js não depende dessas variáveis para compilar — são consumidas em runtime pelos adapters, o que já foi validado deliberadamente.

Segurança / controle de acesso: autenticação via Google OAuth (Supabase Auth) é o único mecanismo hoje — não há SSO corporativo, não há RBAC formal (o produto não tem papéis internos de equipe, é B2C/B2B de rede aberta com um único tipo de sessão autenticada). Identidade do usuário é sempre derivada da sessão no servidor, nunca aceita de input do cliente — esse ponto já foi tratado como fechamento de vulnerabilidade real (spoofing de autor em ações de escrita) numa sessão de hardening documentada.

LGPD/conformidade: não há menção de um programa formal de conformidade LGPD documentado no projeto até o momento — é um ponto em aberto, não resolvido, relevante à medida que o produto sai de ~19 perfis de teste para dados reais de terceiros (creators, marcas, contratos, valores de campanha). Deve ser tratado como requisito a validar na escolha de plataforma (self-hosted vs. gerenciado tem implicações diferentes de responsabilidade sobre dados pessoais brasileiros).

Testes e qualidade: testes de domínio via Vitest, rodando em Node puro, sem banco e sem rede (55+ testes no momento mais recente registrado). `tsc --noEmit` limpo é gate de merge. Checklist de code review em duas camadas: conformidade com spec de produto (ex.: "nenhum score numérico introduzido") e qualidade de arquitetura hexagonal (ex.: "nenhum import de infra em domínio").

Requisito para a plataforma avaliada: precisa ser compatível com deploy serverless/edge gerenciado (Vercel-like) sem exigir operação de infraestrutura própria; precisa suportar gestão de segredos server-only nativa; não há requisito de air-gapped/on-premise.

7. Perfil da Equipe de Desenvolvimento

Quem constrói: fundador único, **Gabriel Quental** — perfil estratégico e de produto, explicitamente não-técnico, publicitário de formação e trajetória (fundador de agência 2020–2023, ex-assessor de creators). Ele não escreve código diretamente.

Como constrói de fato: via **Claude Code** como "equipe de execução" autônoma, mas fortemente governada por documentação viva no próprio repositório — não é vibe coding sem rédea. O padrão de trabalho documentado é: cada sessão de agente lê, nesta ordem,

(`CLAUDE.md`) (estratégia/produto/decisões/quarentenas) → (`AGENTS.md`) (convenções técnicas, estrutura de pastas, sequência obrigatória de construção, checklist de review) → (`docs/decisoões/log.md`) (decisões D1-D11 numeradas, não reabertas sem motivo forte) → o handoff de sessão mais recente em (`docs/sessões/`). Sessões terminam com handoff escrito e push, para permitir trocar de thread/agente sem perder contexto. Existe até uma convenção de compressão de contexto via (`repomix`) para dar contexto de codebase a um agente sem exploração manual arquivo por arquivo.

Implicação direta para a escolha de plataforma: a equipe de desenvolvimento real não é "desenvolvedores full-stack" tradicionais nem "citizen developers" presos a uma interface visual — é um **fundador de produto não-técnico operando um agente de código autônomo contra uma constituição de engenharia versionada em Git**. Isso tem consequências concretas e devem pesar na avaliação:

- A plataforma **precisa expor código-fonte real e versionável em um repositório Git padrão** (GitHub hoje: (`quentalgabriel-cloud/massa-hub-3.0`)) — qualquer solução que prenda lógica de negócio dentro de um builder proprietário sem exportação de código quebra o modo de trabalho atual (agente de código não consegue operar sobre uma "caixa preta" visual da mesma forma).
- A plataforma precisa ser **compreensível e operável por um agente de código genérico** (Claude Code, mas o requisito é mais amplo: qualquer LLM-coding-agent) — frameworks com convenções bem documentadas, TypeScript com tipos fortes, e estrutura de pastas previsível favorecem isso; DSLs proprietárias pouco documentadas ou stacks muito nichadas prejudicam.
- **Não há e não haverá, no horizonte visível, uma equipe de desenvolvedores humanos contratados** — o produto foi desenhado (inclusive nas quarentenas de escopo, seção 4) justamente para não depender de escala de equipe. A avaliação da plataforma deve otimizar para "um agente de IA + um fundador não-técnico consegue manter isso sozinho por trimestres", não para "produtividade de um time de 5 engenheiros".
- O fundador tem um anti-padrão nomeado e ativamente vigiado ("expandir a tese antes de executar a tese") — uma plataforma que incentive adicionar capacidade por estar disponível (ex.: workflow builders, multi-agente, automações extensas) representa um risco de produto documentado, não só uma questão técnica.

Fim do relatório. Fonte: contexto documentado do repositório (`massa-hub-3.0`) (`CLAUDE.md`, `AGENTS.md`, `docs/specs/`, `docs/decisoões/log.md`, `docs/sessões/`, `migrations SQL`) e o briefing de projeto fornecido nesta conversa, junho/2026.